Lộ trình thực hành Cloudflare chuẩn Junior DevOps

Oct 8, 2026 · @Việt Anh

0. Chuẩn bị và cách dùng tài liệu

Trước khi bắt đầu cần đủ 4 thứ: domain, VPS, backend chạy Docker và một repo infra riêng. Mỗi phần gồm 3 bước: **Hiểu cốt lõi** → **Làm** → **Xong khi**. Chưa đạt "Xong khi" thì chưa sang phần sau.

Thứ cần	Yêu cầu tối thiểu
Domain	Loại rẻ (.xyz, .site…), đổi được nameserver
VPS	Ubuntu 22.04/24.04, 1 vCPU, 1–2 GB RAM, có IP public
Backend	Chạy bằng Docker Compose, có `/health`, `/api/auth/login`, một trang admin
Máy local	`dig` (dnsutils), `curl`, `jq`, `git`, Terraform ≥ 1.5, tài khoản GitHub

Quy ước trong tài liệu: domain là `example.com`, API là `api.example.com`, admin là `admin.example.com`. Thay bằng domain của bạn.

Thiết lập kiến trúc đi theo 2 bước. Phần 1–6: DNS trỏ IP VPS qua proxy Cloudflare, Nginx ở origin. Phần 7 trở đi: chuyển sang Tunnel, đóng hoàn toàn port 80/443.

Mỗi phần làm xong, ghi vào `README.md` của repo infra: đã làm gì, gặp lỗi gì, sửa thế nào. File này chính là bằng chứng đưa vào CV.

1. DNS

Mục tiêu: đưa domain về Cloudflare và tự debug được DNS bằng `dig`.

Hiểu cốt lõi

- Đổi nameserver ở nhà đăng ký sang 2 NS mà Cloudflare cấp → Cloudflare trở thành nơi trả lời DNS chính thức cho domain.
- Record cần biết: **A** (tên → IPv4), **CNAME** (tên → tên khác), **TXT** (xác minh, SPF), **MX** (mail), **CAA** (CA nào được cấp cert).

- **Proxied** (đám mây cam): DNS trả IP Cloudflare, traffic đi qua Cloudflare nên có WAF, cache, IP thật bị ẩn. **DNS only** (xám): trả thẳng IP thật.
- Proxy chỉ áp dụng cho HTTP/HTTPS. SSH không đi qua proxy được, sau này sẽ dùng Tunnel/Access.
- TTL: thời gian resolver cache kết quả. Đổi record xong mà chưa thấy thay đổi thường là do cache, không phải cấu hình sai.

Làm

1. Cloudflare Dashboard → Add a domain → gói Free. Kiểm tra lại các record Cloudflare tự quét được.
2. Đổi nameserver ở nhà đăng ký, chờ domain chuyển sang trạng thái Active.
3. Tạo A record `api` → IP VPS, bật Proxied. Tạo thêm `test` → IP VPS, để DNS only.
4. Chạy các lệnh kiểm tra bên dưới, sau đó **xóa record `test`** vì nó làm lộ IP thật.

```
dig NS example.com +short           # phải thấy 2 NS của Cloudflare
dig api.example.com +short          # IP của Cloudflare (proxied)
dig test.example.com +short         # IP thật của VPS (DNS only)
dig @1.1.1.1 api.example.com        # hỏi thẳng resolver cụ thể, xem TTL
curl -sI http://api.example.com | grep -iE "server|cf-ray"
```

Xong khi

- Giải thích được vì sao `api` và `test` trả về IP khác nhau.
- Response có header `server: cloudflare` và `cf-ray`.
- Biết dùng `dig` để phân biệt lỗi do DNS hay lỗi do server.

2. SSL/TLS

Mục tiêu: mã hóa cả hai chặng ở chế độ Full (strict), chuẩn production.

Hiểu cốt lõi

Có hai chặng mã hóa: Client ↔ Cloudflare (Cloudflare tự cấp cert) và Cloudflare ↔ Origin (bạn tự cấu hình). Chế độ SSL quyết định chặng thứ hai.

Chế độ	Cloudflare → Origin	Dùng khi nào
Flexible	HTTP, không mã hóa	Không dùng: chặng sau lộ dữ liệu, dễ bị redirect loop
Full	HTTPS, không kiểm tra cert	Chỉ dùng tạm

Chế độ	Cloudflare → Origin	Dùng khi nào
Full (strict)	HTTPS, cert phải hợp lệ	Chuẩn production

Làm

1. SSL/TLS → Origin Server → Create Certificate (RSA, hostname `example.com` và `*.example.com`). Lưu cert vào `/etc/ssl/cloudflare/origin.pem`, key vào `/etc/ssl/cloudflare/origin.key`, rồi `sudo chmod 600` file key.
2. Cấu hình Nginx (`/etc/nginx/sites-available/api.conf`):

```
server {
    listen 80;
    server_name api.example.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name api.example.com;

    ssl_certificate     /etc/ssl/cloudflare/origin.pem;
    ssl_certificate_key /etc/ssl/cloudflare/origin.key;
    ssl_protocols       TLSv1.2 TLSv1.3;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

3. Bật site: `sudo ln -s /etc/nginx/sites-available/api.conf /etc/nginx/sites-enabled/`, rồi `sudo nginx -t && sudo systemctl reload nginx`.
4. Dashboard: SSL/TLS → Overview → **Full (strict)**. Edge Certificates → bật Always Use HTTPS, Minimum TLS Version = 1.2.
5. HSTS bật sau cùng. Bắt đầu với max-age ngắn (1 ngày), chưa bật includeSubDomains/preload. Bật sai thì trình duyệt người dùng sẽ ép HTTPS cho domain trong suốt thời gian max-age, không gỡ được từ phía server.
6. Nâng cao (tùy chọn): Authenticated Origin Pulls. Nginx yêu cầu client cert của Cloudflare (`ssl_client_certificate` + `ssl_verify_client on`), nhờ vậy chỉ Cloudflare gọi được origin.

```
# Kiểm tra từ máy local
curl -vI https://api.example.com 2>&1 | grep -E "SSL connection|HTTP/"

# Kiểm tra cert origin (chạy trên chính VPS)
openssl s_client -connect 127.0.0.1:443 -servername api.example.com
</dev/null 2>/dev/null \
  | openssl x509 -noout -issuer -enddate
```

Xong khi

- Để Full (strict) mà site vẫn chạy, issuer của cert origin là Cloudflare Origin CA.
- Giải thích được vì sao Flexible cộng với Nginx redirect HTTP → HTTPS gây redirect loop: Cloudflare gọi origin bằng HTTP, Nginx redirect sang HTTPS, Cloudflare lại gọi bằng HTTP, cứ thế lặp mãi.

3. Lấy IP thật của client

Mục tiêu: log và rate limit ở origin dùng đúng IP người dùng, không phải IP Cloudflare.

Hiểu cốt lõi

- Đứng sau proxy, `$remote_addr` trong Nginx là IP của Cloudflare. IP thật nằm trong header `CF-Connecting-IP`.
- Chỉ được tin header này khi request đến từ dải IP của Cloudflare. Nếu tin mọi nguồn, ai cũng có thể tự gửi header giả để đổi IP và vượt rate limit.
- Dải IP của Cloudflare có thể thay đổi, nên phải cập nhật bằng script chạy định kỳ, không chép tay.

Làm

1. Tạo script `/usr/local/bin/update-cf-realip.sh` rồi `sudo chmod +x` nó:

```
#!/usr/bin/env bash
set -euo pipefail
OUT=/etc/nginx/conf.d/cloudflare-realip.conf

{
  echo "# generated $(date -I)"
  for ip in $(curl -fsS https://www.cloudflare.com/ips-v4) $(curl -fsS
https://www.cloudflare.com/ips-v6); do
    echo "set_real_ip_from $ip;"
  done
  echo "real_ip_header CF-Connecting-IP;"
} > "$OUT.new"
```

```
[ -f "$OUT" ] && cp "$OUT" "$OUT.bak"
mv "$OUT.new" "$OUT"

if nginx -t; then
  systemctl reload nginx
else
  echo "Config lỗi, khôi phục bản cũ" >&2
  [ -f "$OUT.bak" ] && mv "$OUT.bak" "$OUT"
  exit 1
fi
```

2. Chạy script một lần: `sudo /usr/local/bin/update-cf-realip.sh`.
3. Trong `location /` của Nginx, chuyển IP thật xuống backend:

```
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

4. Backend cấu hình "trust proxy" chỉ cho 127.0.0.1, đọc IP từ `X-Real-IP`. Mỗi framework có cách riêng, tra theo stack bạn đang dùng.
5. Thêm cron chạy mỗi tuần (`sudo crontab -e`):

```
0 3 * * 1 /usr/local/bin/update-cf-realip.sh >> /var/log/cf-realip.log 2>&1
```

Kiểm tra: tắt wifi, dùng 4G trên điện thoại mở `https://api.example.com/health`. Trên VPS chạy `sudo tail -f /var/log/nginx/access.log`, IP hiện ra phải là IP 4G của bạn.

Xong khi

- Log Nginx và log backend đều ghi đúng IP thật.
- Giải thích được vì sao không được tin `CF-Connecting-IP` từ mọi nguồn.

4. WAF và rate limiting

Mục tiêu: tự viết được rule chặn và rate limit, chứng minh nó hoạt động bằng script.

Hiểu cốt lõi

- **Custom rule** = biểu thức lọc (trường, toán tử, giá trị) cộng hành động (Block, Managed Challenge, Skip…).
- **Rate limiting rule** = đếm request theo một đặc điểm (thường là IP) trong một khoảng thời gian, vượt ngưỡng thì chặn và trả mã 429.
- Gói Free giới hạn số rule và các tùy chọn thời gian, nhưng đủ để thực hành.
- Phòng thủ nhiều lớp: Cloudflare chỉ bảo vệ traffic đi qua nó, nên backend/Nginx vẫn cần rate limit riêng.

Làm

1. Security → WAF → Custom rules. Tạo rule chặn đường dẫn nhạy cảm, action **Block**:

```
(http.request.uri.path contains "/.env") or (http.request.uri.path contains
"/.git") or (http.request.uri.path contains "/wp-admin")
```

2. Tạo rule chặn method lạ vào API, action **Block**:

```
(http.host eq "api.example.com" and not http.request.method in {"GET" "POST"
"PUT" "PATCH" "DELETE" "OPTIONS"})
```

3. Security → WAF → Rate limiting rules. Tạo rule cho endpoint login, đếm theo IP, ví dụ ngưỡng 5 request / 10 giây, action Block:

```
(http.host eq "api.example.com" and http.request.uri.path eq
"/api/auth/login" and http.request.method eq "POST")
```

4. Lớp thứ hai ở Nginx. `limit_req_zone` đặt trong khối `http`, `location` đặt trong `server`. Phần này chỉ đúng khi đã làm xong phần 3, vì nó đếm theo IP thật:

```
limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;

location = /api/auth/login {
    limit_req zone=login burst=5 nodelay;
    limit_req_status 429;
    proxy_pass http://127.0.0.1:8080;
}
```

5. Test bằng script:

```
for i in $(seq 1 20); do
  curl -s -o /dev/null -w "%{http_code}\n" -X POST
https://api.example.com/api/auth/login \
    -H "Content-Type: application/json" \
    -d '{"username":"test","password":"wrong"}'
done

curl -s -o /dev/null -w "%{http_code}\n" https://api.example.com/.env
```

6. Mở Security → Events, tìm các request vừa bị chặn và xem rule nào đã chặn.

Xong khi

- Script login trả vài mã 401 rồi chuyển sang 429.
- `/.env` trả 403.

- Tìm được event tương ứng trong Security Events và đọc được rule nào đã khớp.

5. Khóa origin: chỉ nhận traffic từ Cloudflare

Mục tiêu: gọi thẳng vào IP server thì bị chặn, chỉ đi qua domain mới vào được.

Hiểu cốt lõi

- Ẩn IP là chưa đủ. IP vẫn có thể lộ qua record cũ, header email hay các công cụ quét toàn internet. Server phải tự từ chối kết nối 80/443 không đến từ Cloudflare.
- SSH chỉ mở cho IP của bạn.
- **Bẫy Docker:** khai báo `ports: "8080:8080"` trong Compose sẽ mở port ra internet bằng iptables, **đi vòng qua ufw**. ufw chặn rồi mà port vẫn mở. Luôn bind vào `127.0.0.1`.

Làm

1. Sửa `docker-compose.yml` của backend:

```
services:
  backend:
    ports:
      - "127.0.0.1:8080:8080"
```

2. Cấu hình ufw. Phải cho phép SSH **trước** khi enable, nếu không sẽ tự khóa mình ngoài server. Giữ sẵn web console của nhà cung cấp VPS để phòng khi IP nhà bạn thay đổi:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from <IP_CUA_BAN> to any port 22 proto tcp

for ip in $(curl -fsS https://www.cloudflare.com/ips-v4) $(curl -fsS
https://www.cloudflare.com/ips-v6); do
  sudo ufw allow from "$ip" to any port 80,443 proto tcp
done

sudo ufw enable
sudo ufw status numbered
```

3. Nếu nhà cung cấp VPS có firewall/security group, cấu hình luôn ở đó. Nó chặn từ trước khi traffic tới server nên tốt hơn.

```
# Chạy từ máy local
curl -k --connect-timeout 5 https://<IP_VPS>   # phải timeout
nc -zv -w 5 <IP_VPS> 8080                     # phải thất bại
curl -sI https://api.example.com/health        # vẫn 200
```

Xong khi

- Truy cập thẳng IP bị chặn, đi qua domain vẫn chạy bình thường.
- Port của backend không lộ ra ngoài.
- Giải thích được vì sao Docker có thể đi vòng qua ufw.

6. Zero Trust Access cho trang admin

Mục tiêu: trang admin có một lớp đăng nhập ở Cloudflare, người lạ không bao giờ chạm được tới server.

Hiểu cốt lõi

- Access chặn ngay ở edge của Cloudflare: request chưa xác thực bị dừng lại ở đó, không tới được app. Khác với trang đăng nhập trong app, nơi request đã đến server rồi mới bị kiểm tra.
- Gói Free của Zero Trust đủ dùng cho cá nhân. Khi đăng ký có thể bị yêu cầu thêm phương thức thanh toán dù không bị tính tiền.
- Sau khi xác thực, Cloudflare gửi kèm header `Cf-Access-Jwt-Assertion`. Origin nên kiểm tra JWT này để phòng trường hợp ai đó vòng qua Cloudflare.

Làm

1. Tạo A record `admin` → IP VPS, bật Proxied. Thêm server block Nginx cho `admin.example.com` giống phần 2.
2. Zero Trust dashboard → Settings → Authentication → thêm phương thức **One-time PIN** (mã gửi qua email).
3. Access → Applications → Add an application → **Self-hosted**. Domain: `admin.example.com`, session duration 24h.
4. Tạo policy: Action **Allow**, Include → Emails → email của bạn.
5. Ở backend admin, kiểm tra JWT trong header `Cf-Access-Jwt-Assertion`: chữ ký đúng với public key tại `https://<team-name>.cloudflareaccess.com/cdn-cgi/access/certs`, và trường `aud` khớp với Application Audience (AUD) tag của app. Dùng thư viện JWT có sẵn của ngôn ngữ bạn dùng, không tự viết.

Kiểm tra: mở tab ẩn danh vào `admin.example.com`, phải thấy màn hình đăng nhập của Cloudflare. Thử nhập một email khác, không vào được. Xem lượt đăng nhập trong Zero Trust → Logs → Access.

Xong khi

- Chỉ email của bạn vào được admin.
- Request không có JWT hợp lệ bị backend từ chối.
- Giải thích được Access khác gì so với trang đăng nhập trong app.

7. Cloudflare Tunnel

Mục tiêu: đóng hoàn toàn port 80/443, một tunnel phục vụ nhiều service, server reboot thì tự lên lại.

Hiểu cốt lõi

- `cloudflared` mở kết nối **outbound** từ server đến Cloudflare. Không cần mở port inbound, không cần IP public.
- Kiến trúc thay đổi: bỏ A record trỏ IP, thay bằng CNAME trỏ tới `<TUNNEL_ID>.cfargotunnel.com`. Nginx và Origin Certificate không còn bắt buộc, vì kết nối tunnel đã được mã hóa.
- Chọn tunnel **locally-managed** (cấu hình bằng file `config.yml`) để cấu hình nằm trong git, thay vì chỉ nằm trên dashboard.

Làm

1. Xóa A record `api` và `admin` cũ. Nếu còn, lệnh `route dns` bên dưới sẽ báo lỗi vì trùng tên.
2. Cài `cloudflared` trên VPS rồi tạo tunnel:

```
cloudflared tunnel login                        # mở link, chọn domain
cloudflared tunnel create prod-tunnel           # in ra TUNNEL_ID, tạo file
<TUNNEL_ID>.json
cloudflared tunnel route dns prod-tunnel api.example.com
cloudflared tunnel route dns prod-tunnel admin.example.com

mkdir -p ~/app/cloudflared
cp ~/.cloudflared/<TUNNEL_ID>.json ~/app/cloudflared/
```

3. Tạo `~/app/cloudflared/config.yml`:

```
tunnel: <TUNNEL_ID>
credentials-file: /etc/cloudflared/<TUNNEL_ID>.json

ingress:
  - hostname: api.example.com
    service: http://backend:8080
  - hostname: admin.example.com
    service: http://admin:3000
  - service: http_status:404
```

4. `~/app/docker-compose.yml`. Backend và admin không publish port nào, chỉ nói chuyện với `cloudflared` trong mạng nội bộ của Docker:

```
services:
  backend:
    image: your-backend:1.0.0
    restart: unless-stopped

  admin:
    image: your-admin:1.0.0
    restart: unless-stopped

  cloudflared:
    image: cloudflare/cloudflared:<VERSION>   # ghim version cụ thể, không
dùng latest
    command: tunnel --no-autoupdate --config /etc/cloudflared/config.yml run
    volumes:
      - ./cloudflared:/etc/cloudflared:ro
    restart: unless-stopped
    depends_on: [backend, admin]
```

5. Thêm `cloudflared/*.json` vào `.gitignore`. File credentials là bí mật, tuyệt đối không commit.
6. Đóng port 80/443: `sudo ufw status numbered`, xóa các rule 80/443 bằng `sudo ufw delete <số>`, chỉ giữ SSH.

```
docker compose up -d
docker compose logs cloudflared | grep -i "Registered tunnel connection"
curl -sI https://api.example.com/health

sudo reboot
# SSH vào lại:
docker compose ps
curl -sI https://api.example.com/health
```

Xong khi

- Port 80/443 đã đóng mà cả `api` và `admin` vẫn chạy. Admin vẫn năm sau Access.
- Dashboard báo tunnel hoạt động (Healthy).
- Reboot server xong mọi thứ tự lên, không cần chạy tay lệnh nào.

8. Cache

Mục tiêu: static được cache, API và dữ liệu riêng của user không bao giờ bị cache.

Hiểu cốt lõi

- Mặc định Cloudflare chỉ cache file tĩnh theo đuôi (js, css, ảnh…), không cache HTML/JSON.
- Origin điều khiển cache bằng header `Cache-Control`. Junior phải chủ động đặt header này, không phó mặc cho mặc định.
- Lỗi nghiêm trọng nhất là cache nhầm response chứa dữ liệu riêng, khiến user A thấy thông tin của user B.

Làm

1. Đặt header từ backend (hoặc Nginx) theo loại nội dung:

Loại nội dung	Header
Static có hash trong tên (`app.3f9a.js`)	`Cache-Control: public, max-age=31536000, immutable`
API, trang có đăng nhập	`Cache-Control: private, no-store`
HTML công khai	`Cache-Control: public, max-age=60`

2. Caching → Cache Rules: tạo rule **Bypass cache** cho `(http.host eq "api.example.com")`. Đây là lớp bảo hiểm nếu backend quên đặt header.
3. Kiểm tra header `cf-cache-status`:

```
curl -sI https://example.com/assets/app.js | grep -i cf-cache-status   # lần
1: MISS
curl -sI https://example.com/assets/app.js | grep -i cf-cache-status   # lần
2: HIT
curl -sI https://api.example.com/health   | grep -i cf-cache-status   #
BYPASS hoặc DYNAMIC
```

Giá trị	Ý nghĩa
HIT	Lấy từ cache của Cloudflare
MISS	Chưa có trong cache, đã lấy từ origin và lưu lại
EXPIRED	Có trong cache nhưng hết hạn, đã lấy lại từ origin
BYPASS	Bị bỏ qua cache do rule hoặc header của origin
DYNAMIC	Loại nội dung mặc định không được cache

4. Purge: ưu tiên **Purge by URL**. Hạn chế Purge Everything vì nó đẩy toàn bộ traffic về origin cùng lúc. Purge tự động sẽ làm ở phần 10.

Xong khi

- Static HIT từ lần gọi thứ hai.
- API không bao giờ ra HIT.
- Giải thích được khác nhau giữa BYPASS và DYNAMIC.

9. Terraform quản lý Cloudflare

Mục tiêu: mọi thứ đã bấm tay ở phần 1–8 chuyển thành code trong git, review được, rollback được, dựng lại được.

Hiểu cốt lõi

- Vòng làm việc: `init` → `plan` (xem sẽ thay đổi gì) → `apply` (thực hiện). State là file Terraform dùng để nhớ tài nguyên nào do nó quản lý.
- **Drift**: ai đó sửa tay trên dashboard làm code và thực tế lệch nhau. `terraform plan` sẽ phát hiện ra.
- **Lưu ý version:** Cloudflare provider bản 5 đổi tên và cú pháp nhiều resource so với bản 4 (ví dụ `cloudflare_record` thành `cloudflare_dns_record`). Nhiều tutorial cũ vẫn viết theo bản 4. Luôn ghim version và đối chiếu cú pháp trên Terraform Registry đúng version đó.

Làm

1. Tạo API token: My Profile → API Tokens → Create Custom Token. Chỉ cấp quyền trên đúng zone `example.com`: DNS, Zone Settings, WAF/Rules. Thiếu quyền thì Terraform báo lỗi 403, khi đó mới thêm đúng quyền còn thiếu. **Không dùng Global API Key.**
2. Cấu trúc repo:

```
infra/
├── cloudflare/
│   ├── versions.tf
│   ├── variables.tf
│   ├── dns.tf
│   ├── waf.tf
│   └── imports.tf
├── .gitignore        # .terraform/  *.tfstate*  *.tfvars
└── README.md
```

3. `versions.tf` và `variables.tf`. Token đọc từ biến môi trường `CLOUDFLARE_API_TOKEN`, không ghi vào code:

```
terraform {
  required_version = ">= 1.5"
  required_providers {
    cloudflare = {
      source  = "cloudflare/cloudflare"
      version = "~> 5.0"
    }
  }
}

provider "cloudflare" {}

variable "zone_id" {
  type = string
}
```

4. `dns.tf`:

```
resource "cloudflare_dns_record" "api" {
  zone_id = var.zone_id
  name    = "api.example.com"
  type    = "CNAME"
  content = "<TUNNEL_ID>.cfargotunnel.com"
  proxied = true
  ttl     = 1 # 1 = Auto
}
```

5. `waf.tf`. Mỗi zone chỉ có một ruleset cho mỗi phase, nên rule đã tạo tay ở phần 4 phải import vào, không tạo mới:

```
resource "cloudflare_ruleset" "waf_custom" {
  zone_id = var.zone_id
  name    = "default"
  kind    = "zone"
  phase   = "http_request_firewall_custom"

  rules = [{
    description = "Chan duong dan nhay cam"
    expression  = "(http.request.uri.path contains \"/.env\") or
(http.request.uri.path contains \"/.git\")"
    action      = "block"
    enabled     = true
  }]
}
```

6. Import những gì đã tạo tay (`imports.tf`). Lấy ID record bằng API:

```
curl -s -H "Authorization: Bearer $CLOUDFLARE_API_TOKEN" \
  "https://api.cloudflare.com/client/v4/zones/$ZONE_ID/dns_records?
name=api.example.com" \
  | jq -r '.result[0].id'
```

```
import {
  to = cloudflare_dns_record.api
  id = "<ZONE_ID>/<DNS_RECORD_ID>"
}
```

Nhiều resource thì dùng công cụ `cf-terraforming` của Cloudflare để sinh code và lệnh import tự động, rồi đọc lại và dọn dẹp.

7. Remote state: không để file `tfstate` trên máy cá nhân. Dùng HCP Terraform (gói free) để CI/CD ở phần 10 dùng chung state.
8. Chạy:

```
export CLOUDFLARE_API_TOKEN="..."
export TF_VAR_zone_id="..."
terraform init
terraform fmt -check && terraform validate
terraform plan
terraform apply
```

Bài tập drift: xóa record `api` trên dashboard → `terraform plan` phải báo sẽ tạo lại → `apply` để khôi phục. Sửa tay một rule WAF → `plan` phải phát hiện ra.

Xong khi

- Toàn bộ DNS, WAF, rate limit, cache rule đều nằm trong code.
- `terraform plan` trên môi trường đang chạy trả về No changes.
- Khôi phục được cấu hình sau khi bị xóa/sửa tay.

10. CI/CD với GitHub Actions

Mục tiêu: mở PR thì tự chạy plan để review, merge thì tự apply, deploy app xong thì tự purge cache. Không ai bấm tay trên dashboard nữa.

Hiểu cốt lõi

- PR = nơi review thay đổi hạ tầng trước khi nó xảy ra. `apply` chỉ chạy trên nhánh `main`.
- Mỗi token chỉ có đúng quyền nó cần: token Terraform riêng, token purge cache riêng (chỉ quyền Cache Purge).
- Bí mật năm trong GitHub Secrets, không bao giờ năm trong code hay log.

Làm

1. Kết nối state với HCP Terraform: thêm khối `cloud { organization = "..." workspaces { name = "cloudflare" } }` vào trong khối `terraform` của `versions.tf`. Trong HCP, đổi Execution Mode của workspace sang **Local** để lệnh chạy trên runner của GitHub, HCP chỉ lưu state.
2. GitHub → Settings → Secrets and variables → Actions, thêm: `CLOUDFLARE_API_TOKEN`, `CF_PURGE_TOKEN`, `CF_ZONE_ID`, `TF_API_TOKEN` (token của HCP Terraform).
3. Bật branch protection cho `main`: bắt buộc qua PR và bắt buộc job Terraform phải pass.
4. Tạo `.github/workflows/terraform.yml`:

```
name: terraform

on:
  pull_request:
    paths: ["infra/cloudflare/**"]
  push:
    branches: [main]
    paths: ["infra/cloudflare/**"]

jobs:
  terraform:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: infra/cloudflare
    env:
      CLOUDFLARE_API_TOKEN: ${{ secrets.CLOUDFLARE_API_TOKEN }}
```

```
      TF_VAR_zone_id: ${{ secrets.CF_ZONE_ID }}
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with:
          cli_config_credentials_token: ${{ secrets.TF_API_TOKEN }}
      - run: terraform init -input=false
      - run: terraform fmt -check
      - run: terraform validate
      - run: terraform plan -input=false -out=tfplan
      - name: Apply
        if: github.event_name == 'push' && github.ref == 'refs/heads/main'
        run: terraform apply -input=false tfplan
```

5. Thêm bước purge vào cuối workflow deploy app (sau khi đã deploy thành công). Chỉ purge đúng các URL thay đổi:

```
      - name: Purge Cloudflare cache
        run: |
          curl -fsS -X POST "https://api.cloudflare.com/client/v4/zones/${{
secrets.CF_ZONE_ID }}/purge_cache" \
            -H "Authorization: Bearer ${{ secrets.CF_PURGE_TOKEN }}" \
            -H "Content-Type: application/json" \
            --data '{"files":
["https://example.com/","https://example.com/index.html"]}'
```

Kiểm tra: mở PR đổi một rule WAF → vào tab Actions xem output của `plan` → merge → job chạy `apply` → dashboard cập nhật theo.

Xong khi

- PR có plan để review, merge thì tự apply.
- Deploy app xong cache tự được purge.
- Không có token nào xuất hiện trong code hoặc log.

11. Bài tập debug lỗi 52x

Cố tình gây từng lỗi, quan sát, sửa, rồi ghi lại. Nên làm trên kiến trúc của phần 1–6 (A record + Nginx), vì khi dùng Tunnel các lỗi sẽ khác.

Mã	Ý nghĩa	Cách tái hiện	Hướng xử lý
521	Origin từ chối kết nối	`sudo systemctl stop nginx`	Service có chạy không, port có listen không (`ss -tlnp`)

Mã	Ý nghĩa	Cách tái hiện	Hướng xử lý
522	Kết nối tới origin bị timeout	`sudo ufw insert 1 deny 443/tcp`	Firewall, security group, IP trong DNS có đúng không
524	Kết nối được nhưng origin trả lời quá lâu (khoảng 100 giây)	Tạo endpoint `/slow` sleep 120 giây	Tối ưu query; tác vụ dài chuyển sang job nền, client hỏi lại kết quả sau
525	Bắt tay SSL với origin thất bại	Sửa `listen 443 ssl` thành `listen 443` (port 443 trả HTTP thường)	Kiểm tra cấu hình SSL của Nginx, `ssl_protocols`
526	Cert origin không hợp lệ (ở Full strict)	Thay Origin Cert bằng cert tự ký tạo bằng `openssl req -x509`	Dùng Origin CA cert hoặc cert CA công khai, đúng hostname, còn hạn

Với Tunnel: nếu thấy lỗi 530 kèm error 1033, thường là `cloudflared` không chạy hoặc mất kết nối. Xem `docker compose logs cloudflared`.

Quy trình debug chung

1. Ghi lại mã lỗi và Ray ID trên trang lỗi.
2. Gọi thẳng origin, bỏ qua Cloudflare. Chạy trên chính VPS (vì từ ngoài đã bị firewall chặn):

```
curl -vk --resolve api.example.com:443:127.0.0.1
https://api.example.com/health
```

3. Origin chạy tốt → vấn đề nằm giữa Cloudflare và origin (firewall, SSL mode, DNS). Origin lỗi → xem `/var/log/nginx/error.log` và log của app.
4. Gặp 403 lạ → xem Security → Events, có thể WAF đang chặn nhầm.

Mẫu ghi vào README cho mỗi sự cố: Triệu chứng · Nguyên nhân · Cách phát hiện · Cách sửa · Cách phòng tránh. Phỏng vấn junior rất hay hỏi kiểu "kể một lần bạn xử lý sự cố", 5 lỗi này chính là câu trả lời.

12. Checklist hoàn thành và nguồn học

- [ ] DNS: domain Active trên Cloudflare, debug được bằng `dig`
- [ ] SSL/TLS: Full (strict) + Origin Certificate, giải thích được redirect loop
- [ ] IP thật: log ghi đúng IP, script cập nhật dải IP chạy bằng cron
- [ ] WAF + rate limit: script test ra 429, chặn `/.env`
- [ ] Khóa origin: gọi thẳng IP bị chặn, Docker không lộ port

- [ ] Zero Trust: admin có lớp đăng nhập Access, backend kiểm tra JWT
- [ ] Tunnel: đóng port 80/443, reboot tự lên
- [ ] Cache: static HIT, API không bao giờ HIT
- [ ] Terraform: plan trả No changes, khôi phục được drift
- [ ] CI/CD: PR có plan, merge tự apply, deploy tự purge cache
- [ ] Debug: đã tái hiện và ghi README cho 521, 522, 524, 525, 526

Nguồn học chính (YouTube để hiểu ý tưởng, docs để cấu hình đúng):

Phần	Nguồn
1, 7	Cloudflare One – Video tutorials (video chính thức, ngắn)
7	Video YouTube của NetworkChuck: "EXPOSE your home network to the INTERNET!! (it's safe)"
7	Cloudflare Tunnel – Tutorials (chạy như service, deploy bằng Terraform)
6	BikeGremlin – Cloudflare Access Zero Trust tutorial
9, 10	Techno Tim – Automate Cloudflare with Terraform and GitHub Actions (bài kèm video YouTube)
9	Cloudflare – Terraform tutorial, 6 bài
2, 3, 4, 8, 11	Docs Cloudflare, tìm theo tên mục: SSL/TLS encryption modes · Restoring original visitor IPs · WAF custom rules · Rate limiting rules · Cache Rules · Troubleshooting Cloudflare 5XX errors

Lưu ý: video của Techno Tim và nhiều tutorial cũ dùng Cloudflare provider bản 4. Học ý tưởng từ đó, còn cú pháp thì đối chiếu Terraform Registry theo đúng version bạn đã ghim.